NAME: DARSHAN PRAJAPATI
ROLL:24
MCA-3

SUB:DEEP LEARNING PRACTICAL

1.Implement a simple linear regression model from scratch using Python. Train it on a small dataset and evaluate its performance. Dataset: Boston Housing Dataset

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 1. Load Boston Housing Dataset
url = "https://lib.stat.cmu.edu/datasets/boston"

raw_df =
  pd.read_csv( url,
  sep=r"\s+",
  skiprows=22,
  header=None
)

# Prepare dataset
data = np.hstack([
  raw_df.values[::2, :],
  raw_df.values[1::2, :2]
```

```
])

target = raw_df.values[1::2, 2]

# 2. Select input and output
# RM = Average number of rooms
# MEDV = Median house value (in $1000s)

X = data[:,
5] y =
target

# 3. Split dataset into training and testing
data np.random.seed(42)

indices =
np.random.permutation(len(X)) split =
int(0.8 * len(X))

train_indices =
indices[:split] test_indices
= indices[split:]

X_train = X[train_indices]
X_test = X[test_indices]
```

```
y_train = y[train_indices]

y_test = y[test_indices]
```

```
# 4. Simple Linear Regression from scratch
class SimpleLinearRegression:

    def fit(self, X, y):

        # Calculate means
        x_mean = np.mean(X)
        y_mean = np.mean(y)

        # Calculate slope
        numerator = np.sum(
            (X - x_mean) * (y - y_mean)
        )

        denominator =
            np.sum( (X -
            x_mean) ** 2
        )

        self.slope = numerator / denominator

        # Calculate intercept
        self.intercept = (
            y_mean - self.slope * x_mean
        )
```

```
    def predict(self, X):

        return self.intercept + self.slope * X


# 5. Train the model
model = SimpleLinearRegression()

model.fit(X_train, y_train)

print("Model Training Completed!")
print("Slope:", round(model.slope, 4))
print("Intercept:", round(model.intercept, 4))

# 6. Predict house prices
y_pred =
model.predict(X_test)

# 7. Evaluate model performance

# Mean Squared Error
mse = np.mean((y_test - y_pred) ** 2)

# Root Mean Squared Error
rmse = np.sqrt(mse)
```

```
# Mean Absolute Error
mae = np.mean(np.abs(y_test - y_pred))

# R-squared score
ss_res = np.sum((y_test - y_pred) ** 2)

ss_total = np.sum(
    (y_test - np.mean(y_test)) ** 2
)

r2 = 1 - (ss_res / ss_total)

# 8. Display evaluation results
print("\nModel Evaluation Results")
print("-------------")
print("MSE:", round(mse, 4))
print("RMSE:", round(rmse, 4))
print("MAE:", round(mae, 4))
print("R2 Score:", round(r2, 4))

# 9. Display actual and predicted values
print("\nActual vs Predicted Values")
print("-----------------------")
```

```
for actual, predicted in zip(y_test[:10], y_pred[:10]):
    print(
        f"Actual: {actual:.2f}, "
        f"Predicted:
        {predicted:.2f}"
    )

# 10. Plot actual data and regression line
plt.figure(figsize=(8, 5))

plt.scatter(
    X_test,
    y_test,
    label="Actual Values"
)

sorted_indices = np.argsort(X_test)
X_sorted = X_test[sorted_indices]
y_sorted_pred =
model.predict(X_sorted)

plt.plot(
    X_sorted,
    y_sorted_pre
    d,
    color="red",
```

```
    label="Regression Line"
)
```

```
plt.xlabel("Average Number of Rooms (RM)")
plt.ylabel("Median House Value ($1000s)")
plt.title("Simple Linear Regression - Boston
Housing")

plt.legend()
plt.grid(True)
plt.show()
```

. 2. Build a basic feed-forward neural network using a deep learning framework (e.g., TensorFlow or PyTorch). Train it on the MNIST dataset and report the accuracy. Dataset: MNIST Dataset

```
import torch
import torch.nn as nn
import torch.optim as
optim
from torchvision import datasets,
transforms from torch.utils.data import
DataLoader

# 1. Set device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)
```

```
# 2. Load MNIST dataset

transform =

transforms.ToTensor()
```

```
train_dataset = datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_loader = DataLoader(
    train_dataset, batch_size=64, shuffle=True
)

test_loader = DataLoader(
    test_dataset, batch_size=64, shuffle=False
)

# 3. Create feed-forward neural
network class
NeuralNetwork(nn.Module):
```

```
  def __init__(self):
    super().__init__()

      self.network = nn.Sequential(
        nn.Flatten(),
        nn.Linear(28 * 28,
        128), nn.ReLU(),
        nn.Linear(128, 64),
        nn.ReLU(),
        nn.Linear(64, 10)
      )

 def forward(self, x):
      return self.network(x)



model = NeuralNetwork().to(device)

# 4. Define loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 5. Train the model
epochs = 5
```

```
        for epoch in
    range(epochs):


model.train()

total_loss = 0


for images, labels in train_loader:


   images = images.to(device)

   labels = labels.to(device)


   # Forward propagation

   outputs =

   model(images)

   loss = criterion(outputs, labels)


   # Backpropagation

   optimizer.zero_grad()

   loss.backward()

   optimizer.step()


 total_loss += loss.item()


avg_loss = total_loss / len(train_loader)
```

print(

```
        f"Epoch [{epoch + 1}/{epochs}], "
        f"Loss: {avg_loss:.4f}"
    )

# 6. Evaluate the model
model.eval()

correct = 0
total = 0

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        predictions = torch.argmax(outputs, dim=1)

        total += labels.size(0)
        correct += (predictions == labels).sum().item()

accuracy = 100 * correct / total
```

```
print("\nMNIST Test
Results") print("----")
print("Correct Predictions:", correct)
print("Total Test Images:", total)
print(f"Test Accuracy:
{accuracy:.2f}%")
```

3. Create a synthetic dataset and demonstrate the concepts of bias and variance by training models with varying complexity (e.g., polynomial regression of different degrees). Plot the training and validation errors. Dataset: Synthetic Dataset

```
import torch
import torch.nn as nn
import torch.optim as
optim
from torchvision import datasets,
transforms from torch.utils.data import
DataLoader

# 1. Set device
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

# 2. Load MNIST dataset
transform =
transforms.ToTensor()
```

```
train_dataset = datasets.MNIST(
    root="./data",
```

```
    train=True,
    download=True,
    transform=transform
)

test_dataset = datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_loader = DataLoader(
    train_dataset, batch_size=64, shuffle=True
)

test_loader = DataLoader(
    test_dataset, batch_size=64, shuffle=False
)

# 3. Create feed-forward neural
network class
NeuralNetwork(nn.Module):

    def __init__(self):
        super().__init__()
```

```
        self.network = nn.Sequential(
            nn.Flatten(),
            nn.Linear(28 * 28,
            128), nn.ReLU(),
            nn.Linear(128, 64),
            nn.ReLU(),
            nn.Linear(64, 10)
        )

    def forward(self, x):
        return self.network(x)


model = NeuralNetwork().to(device)

# 4. Define loss function and optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 5. Train the model
epochs = 5

for epoch in range(epochs):
```

```
model.train()
total_loss = 0

for images, labels in train_loader:

    images = images.to(device)
    labels = labels.to(device)

    # Forward propagation
    outputs =
    model(images)
    loss = criterion(outputs, labels)

    # Backpropagation
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    total_loss += loss.item()

avg_loss = total_loss / len(train_loader)

print(
    f"Epoch [{epoch + 1}/{epochs}], "
    f"Loss: {avg_loss:.4f}"
)
```

```
# 6. Evaluate the model
model.eval()

correct = 0
total = 0

with torch.no_grad():
    for images, labels in test_loader:
        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)
        predictions = torch.argmax(outputs, dim=1)

        total += labels.size(0)
        correct += (predictions == labels).sum().item()

accuracy = 100 * correct / total

print("\nMNIST Test Results")
print("---")
print("Correct Predictions:", correct)
```

```
print("Total Test Images:", total)
print(f"Test Accuracy:
{accuracy:.2f}%")
```

4. Generate a high-dimensional dataset and apply Principal Component Analysis (PCA) to reduce its dimensionality. Visualize the data before and after dimensionality reduction.

Dataset: Iris Dataset

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.preprocessing import
StandardScaler from sklearn.decomposition
import PCA

# 1. Load Iris dataset
iris = load_iris()
X = iris.data
y =
iris.target

print("Original dataset shape:", X.shape)

# 2. Standardize the features
scaler = StandardScaler()
X_scaled =
scaler.fit_transform(X)
```

```
# 3. Apply PCA: reduce 4 dimensions to

2 pca = PCA(n_components=2)
```

```
X_pca = pca.fit_transform(X_scaled)


print("Reduced dataset shape:", X_pca.shape)
print("Explained variance ratio:",
pca.explained_variance_ratio_) print(
   "Total variance retained:",
   round(sum(pca.explained_variance_ratio_) * 100,
   2), "%"
)


# 4. Visualize original data using two original
features plt.figure(figsize=(12, 5))


plt.subplot(1, 2, 1)
for i, name in
   enumerate(iris.target_names):
   plt.scatter(
      X[y == i, 0],
      X[y == i, 1],
      label=name
   )


plt.xlabel("Sepal Length (cm)")
plt.ylabel("Sepal Width (cm)")
plt.title("Original Data: Two Original
Features") plt.legend()
```

```
# 5. Visualize reduced data
plt.subplot(1, 2, 2)
for i, name in enumerate(iris.target_names):
    plt.scatter(
        X_pca[y == i,
        0], X_pca[y ==
        i, 1],
        label=name
    )

plt.xlabel("Principal Component 1")
plt.ylabel("Principal Component 2")
plt.title("PCA: Reduced to 2 Dimensions")
plt.legend()

plt.tight_layout()
plt.show()
```

5. Implement a vanilla MLP from scratch using NumPy. Train it on a small dataset and compare

its performance with a similar model built using a deep learning framework.

Dataset: CIFAR-10 Dataset

```
import numpy as np
import matplotlib.pyplot as
```

plt import torch

```
import torch.nn as nn
import torch.optim as
optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, TensorDataset

# 1. Load CIFAR-10
transform = transforms.ToTensor()

train_data = datasets.CIFAR10(
   root="./data",
   train=True,
   download=True,
   transform=transform
)

test_data =
   datasets.CIFAR10(
   root="./data",
   train=False,
   download=True,
   transform=transform
)

# Use a small subset
np.random.seed(42)
```

```
torch.manual_seed(42)
```

```
train_indices = np.random.choice(
   len(train_data), 1000, replace=False
)
test_indices = np.random.choice(
   len(test_data), 300, replace=False
)

X_train = np.stack([
   train_data[i][0].numpy() for i in train_indices
]).reshape(1000, -1)

y_train = np.array([
   train_data[i][1] for i in train_indices
])

X_test = np.stack([
   test_data[i][0].numpy() for i in test_indices
]).reshape(300, -1)

y_test = np.array([
   test_data[i][1] for i in test_indices
])

# 2. NumPy MLP from scratch
```

```
class NumPyMLP:


    def __init__(self, input_size, hidden_size,
        output_size): self.W1 = np.random.randn(
            input_size, hidden_size
        ) * np.sqrt(2 / input_size)
        self.b1 = np.zeros((1, hidden_size))


        self.W2 = np.random.randn(
            hidden_size, output_size
        ) * np.sqrt(2 / hidden_size)
        self.b2 = np.zeros((1, output_size))


    def softmax(self, z):
        z = z - np.max(z, axis=1,
        keepdims=True) exp_z = np.exp(z)
        return exp_z / np.sum(
            exp_z, axis=1, keepdims=True
        )


    def forward(self, X):
        self.z1 = X @ self.W1 + self.b1
        self.a1 = np.maximum(0, self.z1)  # ReLU


        self.z2 = self.a1 @ self.W2 + self.b2
```

```
    self.probs = self.softmax(self.z2)

    return self.probs

def train(self, X, y, epochs=10, lr=0.05,
    batch_size=64): n = len(X)

    for epoch in range(epochs):
        indices = np.random.permutation(n)
        total_loss = 0

        for start in range(0, n, batch_size):
            idx = indices[start:start +
            batch_size] xb, yb = X[idx], y[idx]
            m = len(xb)

            # Forward propagation
            probs = self.forward(xb)

            loss = -np.mean(
                np.log(probs[np.arange(m), yb] + 1e-
                12)
            )
            total_loss += loss * m

            # Backpropagation
```

```
            dz2 = probs.copy()
            dz2[np.arange(m), yb] -=
            1 dz2 /= m

            dW2 = self.a1.T @ dz2
            db2 = np.sum(dz2, axis=0, keepdims=True)

            da1 = dz2 @
            self.W2.T dz1 = da1 *
            (self.z1 > 0)

            dW1 = xb.T @ dz1
            db1 = np.sum(dz1, axis=0, keepdims=True)

            # Gradient descent
            self.W2 -= lr * dW2
            self.b2 -= lr * db2
            self.W1 -= lr * dW1
            self.b1 -= lr * db1

        print(
            f"NumPy Epoch {epoch + 1}/{epochs}, "
            f"Loss: {total_loss / n:.4f}"
        )

def predict(self, X):
```

```
        return np.argmax(self.forward(X), axis=1)



np.random.seed(42)

numpy_model = NumPyMLP(3072, 64, 10)

numpy_model.train(X_train, y_train)


numpy_accuracy = np.mean(

    numpy_model.predict(X_test) == y_test

) * 100


print(f"\nNumPy MLP Test Accuracy: {numpy_accuracy:.2f}%")



# 3. Similar MLP using

PyTorch device =

torch.device(

    "cuda" if torch.cuda.is_available() else "cpu"

)


train_tensor = TensorDataset(

    torch.tensor(X_train,

    dtype=torch.float32),

    torch.tensor(y_train, dtype=torch.long)

)
```

```
train_loader = DataLoader(
```

```
    train_tensor, batch_size=64, shuffle=True
)

torch_model =
    nn.Sequential(
    nn.Linear(3072, 64),
    nn.ReLU(),
    nn.Linear(64, 10)
).to(device)

criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(
    torch_model.parameters(), lr=0.05
)

for epoch in range(10):
    torch_model.train()
    total_loss = 0

    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)

        optimizer.zero_grad()
        outputs =
        torch_model(xb) loss =
        criterion(outputs, yb)
```

```
loss.backward()
```

```
        optimizer.step()

        total_loss += loss.item() * len(xb)

    print(
        f"PyTorch Epoch {epoch + 1}/10, "
        f"Loss: {total_loss / len(train_tensor):.4f}"
    )

# 4. Evaluate PyTorch
model torch_model.eval()
with torch.no_grad():
    X_test_tensor = torch.tensor(
        X_test, dtype=torch.float32
    ).to(device)

    outputs = torch_model(X_test_tensor)
    predictions = outputs.argmax(dim=1).cpu().numpy()

torch_accuracy = np.mean(predictions == y_test) * 100

print("\nFinal
Comparison") print("   ")
print(f"NumPy MLP Accuracy:  {numpy_accuracy:.2f}%")
print(f"PyTorch MLP Accuracy: {torch_accuracy:.2f}%")
```

```
# 5. Plot comparison
plt.bar(
   ["NumPy MLP", "PyTorch
   MLP"], [numpy_accuracy,
   torch_accuracy]
)
plt.ylabel("Test Accuracy (%)")
plt.title("MLP Performance Comparison on CIFAR-
10") plt.show()
```

6. Implement the backpropagation algorithm for a simple neural network from scratch. Verify its correctness by comparing the gradients with those computed using automatic differentiation in a deep learning framework. Dataset: Iris Dataset

```
import numpy as np
import torch
import torch.nn.functional as F
from sklearn.datasets import load_iris

# 1. Load Iris dataset
iris = load_iris()
X =
iris.data.astype(np.float64)
y = iris.target

# Standardize features
```

```
X = (X - X.mean(axis=0)) / X.std(axis=0)
```

```
# Use one sample to verify

gradients x = X[:1]

target = int(y[0])


# 2. Initialize network parameters

np.random.seed(42)


W1 = np.random.randn(4, 5) *

0.1 b1 = np.zeros((1, 5))


W2 = np.random.randn(5, 3) *

0.1 b2 = np.zeros((1, 3))


# 3. NumPy forward

propagation z1 = x @ W1 + b1

a1 = np.maximum(0, z1)


z2 = a1 @ W2 + b2


# Softmax

exp_z = np.exp(z2 - np.max(z2, axis=1,

keepdims=True)) probs = exp_z / np.sum(exp_z,

axis=1, keepdims=True)


loss = -np.log(probs[0, target])
```

```
print("NumPy Loss:", loss)

# 4. NumPy
backpropagation dz2 =
probs.copy()
dz2[0, target] -= 1

dW2 = a1.T @ dz2
db2 = np.sum(dz2, axis=0, keepdims=True)

da1 = dz2 @ W2.T
dz1 = da1 * (z1 > 0)

dW1 = x.T @ dz1
db1 = np.sum(dz1, axis=0, keepdims=True)

# 5. PyTorch automatic differentiation
dtype = torch.float64

xt = torch.tensor(x, dtype=dtype)
yt = torch.tensor([target], dtype=torch.long)

W1t = torch.tensor(W1, dtype=dtype,
requires_grad=True) b1t = torch.tensor(b1,
dtype=dtype, requires_grad=True)
```

```
W2t = torch.tensor(W2, dtype=dtype,

requires_grad=True) b2t = torch.tensor(b2,

dtype=dtype, requires_grad=True)


z1t = xt @ W1t + b1t

a1t = torch.relu(z1t)


z2t = a1t @ W2t + b2t


loss_t = F.cross_entropy(z2t, yt)

loss_t.backward()


# 6. Compare gradients

print("\nGradient

Comparison") print("   ")


print("NumPy dW1:\n", dW1)

print("PyTorch dW1:\n",

W1t.grad.numpy())


print("\nNumPy db1:\n", db1)

print("PyTorch db1:\n",

b1t.grad.numpy())


print("\nNumPy dW2:\n", dW2)

print("PyTorch dW2:\n",
```

```
W2t.grad.numpy())


print("\nNumPy db2:\n", db2)
```

```
print("PyTorch db2:\n", b2t.grad.numpy())


# 7. Verify numerical agreement
print("\nMaximum Absolute Gradient
Differences") print(
   "dW1:",
   np.max(np.abs(dW1 - W1t.grad.numpy()))
)
print(
   "db1:",
   np.max(np.abs(db1 - b1t.grad.numpy()))
)
print(
   "dW2:",
   np.max(np.abs(dW2 - W2t.grad.numpy()))
)
print(
   "db2:",
   np.max(np.abs(db2 - b2t.grad.numpy()))
)


# 8. Check correctness
assert np.allclose(dW1, W1t.grad.numpy(),
atol=1e-8) assert np.allclose(db1,
b1t.grad.numpy(), atol=1e-8) assert
np.allclose(dW2, W2t.grad.numpy(), atol=1e-8)
```

```
assert np.allclose(db2, b2t.grad.numpy(), atol=1e-8)

print("\nSUCCESS: NumPy gradients match PyTorch gradients!")
```

7. Design and train a neural network to approximate a given continuous function (e.g., sine function). Experiment with different network architectures and report the results.

Dataset: Sine Function Dataset

```
import numpy as np
import matplotlib.pyplot as
plt import torch
import torch.nn as nn
import torch.optim as
optim

# 1. Create sine function dataset
np.random.seed(42)
torch.manual_seed(42)

X = np.linspace(-2 * np.pi, 2 * np.pi, 500).reshape(-
1, 1) y = np.sin(X)

# Convert to PyTorch tensors
X_tensor = torch.tensor(X, dtype=torch.float32)
y_tensor = torch.tensor(y, dtype=torch.float32)

# 2. Define neural network
```

```
class SineNetwork(nn.Module):

    def __init__(self, hidden_size):
        super().__init__()

        self.network = nn.Sequential(
            nn.Linear(1, hidden_size),
            nn.Tanh(),
            nn.Linear(hidden_size, hidden_size),
            nn.Tanh(),
            nn.Linear(hidden_size, 1)
        )

    def forward(self, x):
        return self.network(x)



# 3. Train different
architectures architectures =
[8, 32, 64] results = {}

for hidden_size in architectures:

    model = SineNetwork(hidden_size)
```

```
criterion = nn.MSELoss()
optimizer = optim.Adam(
    model.parameters(), lr=0.01
)

# Training
for epoch in range(1000):

    predictions = model(X_tensor)
    loss = criterion(predictions, y_tensor)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 200 == 0:
        print(
            f"Hidden neurons: {hidden_size}, "
            f"Epoch: {epoch + 1}, "
            f"Loss: {loss.item():.6f}"
        )

# Evaluate training error
model.eval()
```

```
    with torch.no_grad():
        predicted_y = model(X_tensor).numpy()

    mse = np.mean((y - predicted_y) ** 2)

    results[hidden_size] = {
        "predictions":
        predicted_y, "mse": mse
    }

# 4. Plot actual and predicted functions
plt.figure(figsize=(10, 6))

plt.plot(
    X, y,
    label="Actual sin(x)",
    linewidth=3
)

for hidden_size in architectures:
    plt.plot(
        X,
        results[hidden_size]["predictions"],
        label=f"Hidden neurons:
        {hidden_size}"
    )
```

```
plt.xlabel("x")
plt.ylabel("y")
plt.title("Neural Network Approximation of Sine Function")
plt.legend()
plt.grid(True)
plt.show()

# 5. Report results
print("\nArchitecture Comparison") print("")

for hidden_size in architectures:
    print(
        f"Hidden neurons: {hidden_size}, "
        f"Training MSE: {results[hidden_size]['mse']:.8f}"
    )

# 6. Plot MSE comparison
plt.figure(figsize=(8, 5))

plt.bar(
    [str(size) for size in architectures],
    [results[size]["mse"] for size in architectures]
)
```

```
plt.xlabel("Number of Hidden Neurons per
Layer") plt.ylabel("Mean Squared Error")
plt.title("Sine Approximation Error by
Architecture") plt.show()
```

8. Train a neural network on a dataset with raw features and another network on the same dataset with engineered features. Compare the performance of both models and discuss the impact of feature representation. Dataset: Titanic Dataset

```
import numpy as np
import tensorflow as
tf
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import
fashion_mnist from tensorflow.keras.models
import Sequential from tensorflow.keras.layers
import (
   Input, Flatten, Dense, Dropout
)
from tensorflow.keras.optimizers import Adam

# 1. Load Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()
```

```
print("Training data shape:", X_train.shape)
```

```
print("Testing data shape:", X_test.shape)

# 2. Normalize pixel values from 0-255 to
0-1 X_train = X_train.astype("float32") /
255.0 X_test = X_test.astype("float32") /
255.0

# 3. Define hyperparameter choices
hidden_units_options = [64, 128, 256]
learning_rate_options = [0.001, 0.0005]
dropout_options = [0.2, 0.4]

# 4. Random search settings
np.random.seed(42)
tf.keras.utils.set_random_seed(42)

number_of_trials = 6
results = []

best_val_accuracy = -1
best_model = None
best_params = None

# 5. Perform random search
for trial in range(number_of_trials):
```

```
hidden_units = int(
    np.random.choice(hidden_units_optio
    ns)
)

learning_rate = float(
    np.random.choice(learning_rate_optio
    ns)
)

dropout_rate = float(
    np.random.choice(dropout_options)
)

print(f"\nTrial {trial + 1}/{number_of_trials}")
print("Hidden units:", hidden_units)
print("Learning rate:", learning_rate)
print("Dropout rate:", dropout_rate)

# Build model
model = Sequential([
    Input(shape=(28,
    28)), Flatten(),
    Dense(hidden_units,
    activation="relu"),
    Dropout(dropout_rate),
```

```
Dense(64, activation="relu"),
Dense(10,
activation="softmax")
```

```
])

# Compile model
model.compile(
    optimizer=Adam(learning_rate=learning_rate)
    , loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Train model
history =
model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=0
)

# Best validation accuracy during this
trial trial_val_accuracy = max(
    history.history["val_accuracy"]
)

print(
```

```
    f"Best validation accuracy: "
    f"{trial_val_accuracy * 100:.2f}%"
)

results.append({
    "trial": trial + 1,
    "hidden_units": hidden_units,
    "learning_rate": learning_rate,
    "dropout": dropout_rate,
    "val_accuracy":
    trial_val_accuracy
})

# Save the best model
if              trial_val_accuracy              >
    best_val_accuracy:
    best_val_accuracy                       =
    trial_val_accuracy    best_model    =
    model
    best_params = {
        "hidden_units":
        hidden_units,
        "learning_rate":
        learning_rate,
        "dropout_rate":
        dropout_rate
```

```
    }


# 6. Display all trial
results print("\nAll Trial
Results¯¯¯
```

```
for result in results:
    print(result)

# 7. Display best
hyperparameters print("\nBest
Hyperparameters:") print("      ")
print("Hidden units:",
best_params["hidden_units"]) print("Learning
rate:", best_params["learning_rate"])
print("Dropout rate:",
best_params["dropout_rate"]) print(
    "Best validation accuracy:",
    round(best_val_accuracy * 100, 2), "%"
)

# 8. Evaluate the best model on the test set
test_loss, test_accuracy = best_model.evaluate(
    X_test, y_test, verbose=0
)

print("\nFinal Test
Performance:") print(" ")
print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
```

```
# 9. Plot validation accuracy for each trial
trial_numbers = [
   result["trial"] for result in results
]

validation_accuracies = [
   result["val_accuracy"] * 100 for result in results
]

plt.figure(figsize=(8, 5))

plt.plot(
   trial_numbers,
   validation_accuracies,
   marker="o"
)

plt.xlabel("Random Search Trial")
plt.ylabel("Best Validation Accuracy (%)")
plt.title("Hyperparameter Tuning - Fashion
MNIST") plt.xticks(trial_numbers)
plt.grid(True)
plt.show()

# 10. Show a sample prediction
```

```
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress",
    "Coat", "Sandal", "Shirt", "Sneaker",
    "Bag", "Ankle boot"
]

sample_images = X_test[:5]
sample_predictions = best_model.predict(
    sample_images, verbose=0
).argmax(axis=1)

plt.figure(figsize=(10, 3))

for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(sample_images[i], cmap="gray")
    plt.title(class_names[sample_predictions[i]])
    plt.axis("off")

plt.tight_layout()
plt.show()
```

9. Implement L2 regularization in a neural network and train it on a dataset. Compare the performance with a model without regularization and discuss the effects. Dataset: Boston Housing Dataset

```
import numpy as np
import tensorflow as
tf
import matplotlib.pyplot as plt

from tensorflow.keras.datasets import
fashion_mnist from tensorflow.keras.models
import Sequential from tensorflow.keras.layers
import (
   Input, Flatten, Dense, Dropout
)
from tensorflow.keras.optimizers import Adam

# 1. Load Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

print("Training data shape:",
X_train.shape) print("Testing data
shape:", X_test.shape)

# 2. Normalize pixel values from 0-255 to
0-1 X_train = X_train.astype("float32") /
255.0 X_test = X_test.astype("float32") /
255.0

# 3. Define hyperparameter choices
```

```
hidden_units_options = [64, 128, 256]

learning_rate_options = [0.001, 0.0005]

dropout_options = [0.2, 0.4]
```

```
# 4. Random search settings
np.random.seed(42)
tf.keras.utils.set_random_seed(42)

number_of_trials = 6
results = []

best_val_accuracy = -1
best_model = None
best_params = None

# 5. Perform random search
for trial in range(number_of_trials):

  hidden_units = int(
    np.random.choice(hidden_units_optio
    ns)
  )

  learning_rate = float(
    np.random.choice(learning_rate_optio
    ns)
  )

  dropout_rate = float(
```

```
np.random.choice(dropout_options)
```

```
)

print(f"\nTrial {trial + 1}/{number_of_trials}")
print("Hidden units:", hidden_units)
print("Learning rate:", learning_rate)
print("Dropout rate:", dropout_rate)

# Build model
model = Sequential([
   Input(shape=(28,
   28)), Flatten(),
   Dense(hidden_units,
   activation="relu"),
   Dropout(dropout_rate),
   Dense(64, activation="relu"),
   Dense(10,
   activation="softmax")
])

# Compile model
model.compile(
   optimizer=Adam(learning_rate=learning_rate)
   , loss="sparse_categorical_crossentropy",
   metrics=["accuracy"]
)
```

Train model

```
history =
    model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128
    ,
    validation_split=0.1,
    verbose=0
)

# Best validation accuracy during this
trial trial_val_accuracy = max(
    history.history["val_accuracy"]
)

print(
    f"Best validation accuracy: "
    f"{trial_val_accuracy * 100:.2f}%"
)

results.append({
    "trial": trial + 1,
    "hidden_units": hidden_units,
    "learning_rate": learning_rate,
    "dropout": dropout_rate,
```

"val_accuracy":

trial_val_accuracy

```
    })

    # Save the best model
    if trial_val_accuracy >
        best_val_accuracy:
        best_val_accuracy =
        trial_val_accuracy best_model =
        model
        best_params = {
            "hidden_units":
            hidden_units,
            "learning_rate":
            learning_rate,
            "dropout_rate":
            dropout_rate
        }

# 6. Display all trial
results print("\nAll Trial
Results") print("-------")

for result in results:
    print(result)

# 7. Display best
hyperparameters print("\nBest
-------------
```

```
Hyperparameters") print("      ")

print("Hidden units:",

best_params["hidden_units"]) print("Learning

rate:", best_params["learning_rate"])

print("Dropout rate:",

best_params["dropout_rate"])
```

```
print(
   "Best validation accuracy:",
   round(best_val_accuracy * 100, 2), "%"
)

# 8. Evaluate the best model on the test set
test_loss, test_accuracy = best_model.evaluate(
   X_test, y_test, verbose=0
)

print("\nFinal Test
Performance") print(" ")
print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")

# 9. Plot validation accuracy for each trial
trial_numbers = [
   result["trial"] for result in results
]

validation_accuracies = [
   result["val_accuracy"] * 100 for result in results
]

plt.figure(figsize=(8, 5))
```

```
plt.plot(
    trial_numbers,
    validation_accuracies,
    marker="o"
)

plt.xlabel("Random Search Trial")
plt.ylabel("Best Validation Accuracy (%)")
plt.title("Hyperparameter Tuning - Fashion
MNIST") plt.xticks(trial_numbers)
plt.grid(True)
plt.show()

# 10. Show a sample prediction
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress",
    "Coat", "Sandal", "Shirt", "Sneaker",
    "Bag", "Ankle boot"
]

sample_images = X_test[:5]
sample_predictions = best_model.predict(
    sample_images, verbose=0
).argmax(axis=1)
```

```
plt.figure(figsize=(10, 3))


for i in range(5):
   plt.subplot(1, 5, i + 1)
   plt.imshow(sample_images[i], cmap="gray")
   plt.title(class_names[sample_predictions[i]])
   plt.axis("off")


plt.tight_layout()
plt.show()
```

10. Perform hyperparameter tuning for a neural network using grid search or random search. Train the model on a dataset and report the best hyperparameters and corresponding performance.

Dataset: Fashion MNIST Dataset

```
import numpy as np
import tensorflow as
tf
import matplotlib.pyplot as plt


from tensorflow.keras.datasets import
fashion_mnist from tensorflow.keras.models
import Sequential from tensorflow.keras.layers
import (
   Input, Flatten, Dense, Dropout
```

)

from tensorflow.keras.optimizers import Adam

```
# 1. Load Fashion MNIST dataset

(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()


print("Training data shape:",

X_train.shape) print("Testing data

shape:", X_test.shape)


# 2. Normalize pixel values from 0-255 to

0-1 X_train = X_train.astype("float32") /

255.0 X_test = X_test.astype("float32") /

255.0


# 3. Define hyperparameter choices

hidden_units_options = [64, 128, 256]

learning_rate_options = [0.001, 0.0005]

dropout_options = [0.2, 0.4]


# 4. Random search settings

np.random.seed(42)

tf.keras.utils.set_random_seed(42)


number_of_trials = 6

results = []


best_val_accuracy = -1
```

```
best_model = None
```

```
best_params = None

# 5. Perform random search
for trial in range(number_of_trials):

    hidden_units = int(
        np.random.choice(hidden_units_optio
        ns)
    )

    learning_rate = float(
        np.random.choice(learning_rate_optio
        ns)
    )

    dropout_rate = float(
        np.random.choice(dropout_options)
    )

    print(f"\nTrial {trial + 1}/{number_of_trials}")
    print("Hidden units:", hidden_units)
    print("Learning rate:", learning_rate)
    print("Dropout rate:", dropout_rate)

    # Build model
    model = Sequential([
```

```
Input(shape=(28,

28)),
```

```
    Flatten(),
    Dense(hidden_units,
    activation="relu"),
    Dropout(dropout_rate),
    Dense(64, activation="relu"),
    Dense(10,
    activation="softmax")
])

# Compile model
model.compile(
    optimizer=Adam(learning_rate=learning_rate)
    , loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Train model
history =
model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=0
)
```

Best validation accuracy during this trial

```
trial_val_accuracy = max(
    history.history["val_accuracy"]
)

print(
    f"Best validation accuracy: "
    f"{trial_val_accuracy * 100:.2f}%"
)

results.append({
    "trial": trial + 1,
    "hidden_units": hidden_units,
    "learning_rate": learning_rate,
    "dropout": dropout_rate,
    "val_accuracy":
    trial_val_accuracy
})

# Save the best model
if              trial_val_accuracy              >
    best_val_accuracy:
    best_val_accuracy                       =
    trial_val_accuracy    best_model    =
    model
    best_params = {
        "hidden_units":
```

hidden_units,

"learning_rate":

learning_rate,

"dropout_rate":

dropout_rate

```
    }

# 6. Display all trial
results print("\nAll Trial
Results") print("")

for result in results:
  print(result)

# 7. Display best
hyperparameters print("\nBest
Hyperparameters") print("")
print("Hidden units:",
best_params["hidden_units"]) print("Learning
rate:", best_params["learning_rate"])
print("Dropout rate:",
best_params["dropout_rate"]) print(
  "Best validation accuracy:",
  round(best_val_accuracy * 100, 2), "%"
)

# 8. Evaluate the best model on the test set
test_loss, test_accuracy = best_model.evaluate(
  X_test, y_test, verbose=0
)
```

```
print("\nFinal Test Performance") print(" ")
print("Test Loss:", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")

# 9. Plot validation accuracy for each trial
trial_numbers = [
    result["trial"] for result in results
]

validation_accuracies = [
    result["val_accuracy"] * 100 for result in results
]

plt.figure(figsize=(8, 5))

plt.plot(
    trial_numbers,
    validation_accuracies,
    marker="o"
)

plt.xlabel("Random Search Trial")
plt.ylabel("Best Validation Accuracy (%)")
plt.title("Hyperparameter Tuning - Fashion MNIST")
```

```
plt.xticks(trial_numbers)
plt.grid(True)
plt.show()

# 10. Show a sample prediction
class_names = [
    "T-shirt/top", "Trouser", "Pullover", "Dress",
    "Coat", "Sandal", "Shirt", "Sneaker",
    "Bag", "Ankle boot"
]

sample_images = X_test[:5]
sample_predictions = best_model.predict(
    sample_images, verbose=0
).argmax(axis=1)

plt.figure(figsize=(10, 3))

for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(sample_images[i], cmap="gray")
    plt.title(class_names[sample_predictions[i]])
    plt.axis("off")

plt.tight_layout()
```

```
plt.show()
```

ASSIGNMENT-2

1. Implement a basic 2D convolution operation from scratch. Dataset: Use a small set of grayscale images, such as the MNIST dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import mnist

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

# Select one grayscale image
image =
X_train[0].astype(float)

print("Original Image Shape:", image.shape)

# 2. Define a 3x3 convolution kernel
kernel = np.array([
   [1, 0, -1],
  [1, 0, -1],
  [1, 0, -1]
        ],
dtype=float)
```

```
# 3. Implement 2D convolution from
scratch def convolution2d(image, kernel,
stride=1):

    image_h, image_w = image.shape
    kernel_h, kernel_w = kernel.shape

    output_h = (image_h - kernel_h) // stride + 1
    output_w = (image_w - kernel_w) // stride + 1

    output = np.zeros((output_h, output_w))

    for i in range(output_h):
        for j in
        range(output_w):

            # Extract image
            region region =
            image[
                i * stride:i * stride + kernel_h,
                j * stride:j * stride + kernel_w
            ]

            # Element-wise multiplication and
            sum output[i, j] = np.sum(region *
            kernel)
```

return output

```
# 4. Apply convolution
feature_map = convolution2d(image, kernel)

print("Feature Map Shape:", feature_map.shape)

# 5. Visualize results
plt.figure(figsize=(10, 4))

plt.subplot(1, 3, 1)
plt.imshow(image,
cmap="gray")
plt.title("Original MNIST
Image") plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(kernel,
cmap="gray")
plt.title("Convolution Kernel")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(feature_map,
cmap="gray") plt.title("Feature
Map")
plt.axis("off")
```

```
plt.tight_layout()

plt.show()
```

2. Apply different convolution kernels (e.g., edge detection, sharpening) to an image and visualize the results. Dataset: Use a set of sample images from the CIFAR-10 dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import cifar10

# 1. Load CIFAR-10 dataset
(X_train, y_train), (X_test, y_test) = cifar10.load_data()

# Select one RGB image
image = X_train[0].astype(float)

# Normalize pixel values
image = image / 255.0

print("Image Shape:", image.shape)

# 2. Define convolution
operation def
convolution2d(image, kernel):

    h, w = image.shape
    kh, kw =
    kernel.shape
```

```
output = np.zeros((h - kh + 1, w - kw + 1))
```

```
    for i in
        range(output.shape[0]): for j
        in range(output.shape[1]):

            region = image[i:i + kh, j:j + kw]
            output[i, j] = np.sum(region *
            kernel)

    return output

# 3. Convert RGB image to grayscale
gray = (
    0.299 * image[:, :, 0]
    + 0.587 * image[:, :, 1]
    + 0.114 * image[:, :, 2]
)

# 4. Define different kernels

# Edge detection
edge_kernel = np.array([
    [-1, -1, -1],
    [-1,  8, -1],
    [-1, -1, -1]
])
```

```
# Sharpening
sharpen_kernel =
np.array([
   [ 0, -1,  0],
   [-1,  5, -1],
   [ 0, -1,  0]
])

# Blurring
blur_kernel = np.ones((3, 3)) / 9

# 5. Apply kernels
edge_result = convolution2d(gray, edge_kernel)
sharp_result = convolution2d(gray,
sharpen_kernel) blur_result =
convolution2d(gray, blur_kernel)

# 6. Display images
plt.figure(figsize=(12, 7))

plt.subplot(2, 3, 1)
plt.imshow(image)
plt.title("Original CIFAR-10
Image") plt.axis("off")

plt.subplot(2, 3, 2)
```

```
plt.imshow(gray,

cmap="gray")
```

```
plt.title("Grayscale
Image") plt.axis("off")

plt.subplot(2, 3, 3)
plt.imshow(edge_result,
cmap="gray") plt.title("Edge
Detection") plt.axis("off")

plt.subplot(2, 3, 4)
plt.imshow(sharp_result,
cmap="gray") plt.title("Sharpening")
plt.axis("off")

plt.subplot(2, 3, 5)
plt.imshow(np.clip(blur_result, 0, 1),
cmap="gray") plt.title("Blurring")
plt.axis("off")

plt.subplot(2, 3, 6)
plt.imshow(np.abs(edge_result),
cmap="gray") plt.title("Edge Magnitude")
plt.axis("off")

plt.tight_layout()
plt.show()
```

3. Implement max pooling and average pooling operations and compare their effects on an image. Dataset: Use images from the Fashion MNIST dataset.

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.datasets import fashion_mnist

# 1. Load Fashion MNIST dataset
(X_train, y_train), (X_test, y_test) = fashion_mnist.load_data()

# Select one grayscale image
image =
X_train[0].astype(float)

print("Original Image Shape:", image.shape)

# 2. Implement max pooling
def max_pooling(image, pool_size=2, stride=2):

    h, w = image.shape

    output_h = (h - pool_size) // stride + 1
    output_w = (w - pool_size) // stride + 1

    output = np.zeros((output_h, output_w))
```

```
    for i in range(output_h):
        for j in
        range(output_w):

            region = image[
                i * stride:i * stride + pool_size,
                j * stride:j * stride + pool_size
            ]

            output[i, j] = np.max(region)

    return output

# 3. Implement average pooling
def average_pooling(image, pool_size=2, stride=2):

    h, w = image.shape

    output_h = (h - pool_size) // stride + 1
    output_w = (w - pool_size) // stride + 1

    output = np.zeros((output_h, output_w))

    for i in range(output_h):
        for j in
        range(output_w):
```

```
        region = image[
            i * stride:i * stride + pool_size,
            j * stride:j * stride + pool_size
        ]

        output[i, j] = np.mean(region)

    return output

# 4. Apply pooling
max_result = max_pooling(image)
avg_result = average_pooling(image)

print("Max Pooling Shape:", max_result.shape)
print("Average Pooling Shape:",
avg_result.shape)

# 5. Visualize results
plt.figure(figsize=(12, 4))

plt.subplot(1, 3, 1)
plt.imshow(image,
cmap="gray") plt.title("Original
Image (28x28)") plt.axis("off")

plt.subplot(1, 3, 2)
```

```
plt.imshow(max_result,
cmap="gray") plt.title("Max
Pooling (14x14)") plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(avg_result,
cmap="gray") plt.title("Average
Pooling (14x14)") plt.axis("off")

plt.tight_layout()
plt.show()
```

4. Experiment with different stride values in the convolution operation and observe the changes in the output feature maps. Dataset: Use a subset of images from the SVHN dataset.

```
import numpy as np
import matplotlib.pyplot as
plt from scipy.io import
loadmat
from urllib.request import urlretrieve
import os

# 1. Download SVHN
dataset url = (
   "http://ufldl.stanford.edu/housenumber
```

s/" "train_32x32.mat"

```
)
filename = "svhn_train.mat"

if not os.path.exists(filename):
   urlretrieve(url, filename)

# 2. Load a small subset of SVHN
data = loadmat(filename)

images = data["X"]  # Shape: (32, 32, 3, N)
labels = data["y"].flatten()

# Select first 5 images
images = images[:, :, :, :5]
labels = labels[:5]

# Select one image and convert to
grayscale image = images[:, :, :,
0].astype(float)

gray = (
   0.299 * image[:, :, 0]
   + 0.587 * image[:, :, 1]
   + 0.114 * image[:, :, 2]
)
```

```
gray = gray / 255.0

print("Original Image Shape:", gray.shape)
print("Digit Label:", int(labels[0]) % 10)

# 3. Define a convolution kernel
kernel = np.array([
   [-1, 0, 1],
   [-1, 0, 1],
   [-1, 0, 1]
], dtype=float)

# 4. Implement convolution from
scratch def convolution2d(image,
kernel, stride=1):

   h, w = image.shape
   kh, kw =
   kernel.shape

   output_h = (h - kh) // stride + 1
   output_w = (w - kw) // stride + 1

   output = np.zeros((output_h, output_w))

   for i in range(output_h):
```

```
for j in
range(output_w):
```

```
            region = image[
                i * stride:i * stride + kh,
                j * stride:j * stride + kw
            ]

            output[i, j] = np.sum(region * kernel)

    return output

# 5. Apply different stride
values strides = [1, 2, 3, 4]
feature_maps = []

for stride in strides:
    result = convolution2d(gray, kernel,
    stride) feature_maps.append(result)

    print(
        f"Stride {stride}: "
        f"Feature Map Shape = {result.shape}"
    )

# 6. Visualize results
plt.figure(figsize=(12, 8))
```

```
plt.subplot(2, 3, 1)

plt.imshow(gray,

cmap="gray")

plt.title("Original SVHN

Image") plt.axis("off")


for i, stride in enumerate(strides):


   plt.subplot(2, 3, i + 2)

   plt.imshow(feature_maps[i],

   cmap="gray") plt.title(

      f"Stride {stride}\n"

      f"Shape: {feature_maps[i].shape}"

   )

   plt.axis("off")


plt.tight_layout()

plt.show()
```

5. Build a simple CNN using a deep learning framework like TensorFlow or PyTorch and train it on a dataset. Dataset: Use the MNIST dataset for training and testing.

```
import numpy as np

import matplotlib.pyplot as

plt import tensorflow as tf
```

```
from tensorflow.keras.datasets import mnist
from tensorflow.keras.models import
Sequential from tensorflow.keras.layers
import (
   Conv2D, MaxPooling2D, Flatten, Dense, Dropout
)

# 1. Load MNIST dataset
(X_train, y_train), (X_test, y_test) = mnist.load_data()

print("Training images:", X_train.shape)
print("Testing images:", X_test.shape)

# 2. Normalize images
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# Add channel dimension
X_train = X_train[...,
np.newaxis] X_test =
X_test[..., np.newaxis]

# 3. Build CNN
model model =
Sequential([
   tf.keras.layers.Input(shape=(28, 28, 1)),
```

Conv2D(

```
        32, kernel_size=(3, 3),
        activation="relu"
    ),

    MaxPooling2D(pool_size=(2,  2)),

    Conv2D(
        64, kernel_size=(3, 3),
        activation="relu"
    ),

    MaxPooling2D(pool_size=(2,  2)),

    Flatten(),

    Dense(128, activation="relu"),

    Dropout(0.3),

    Dense(10, activation="softmax")
])

# 4. Compile model
model.compile(
    optimizer="adam",
```

```
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

# Display model
architecture
model.summary()

# 5. Train CNN
history =
model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1
)

# 6. Evaluate on test dataset
test_loss, test_accuracy = model.evaluate(
    X_test, y_test, verbose=0
)

print("\nCNN
Evaluation") print("      ")
```

```
print("Test Loss:", round(test_loss, 4))
```

```
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")


# 7. Plot training and validation accuracy

plt.figure(figsize=(10, 4))


plt.subplot(1, 2, 1)

plt.plot(history.history["accuracy"],

label="Training")

plt.plot(history.history["val_accuracy"], label="Validation")

plt.title("Accuracy")

plt.xlabel("Epoch")

plt.ylabel("Accuracy"

) plt.legend()

plt.grid(True)


# 8. Plot training and validation loss

plt.subplot(1, 2, 2)

plt.plot(history.history["loss"],

label="Training")

plt.plot(history.history["val_loss"],

label="Validation") plt.title("Loss")

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.legend()

plt.grid(True)
```

```
plt.tight_layout()
```

```
plt.show()


# 9. Display predictions
predictions = model.predict(X_test[:10],
verbose=0) predicted_labels =
np.argmax(predictions, axis=1)


plt.figure(figsize=(12, 4))


for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(X_test[i].squeeze(),
    cmap="gray") plt.title(
        f"True: {y_test[i]}\nPred: {predicted_labels[i]}"
    )
    plt.axis("off")


plt.tight_layout()
plt.show()
```

6. Implement and compare the performance of standard convolution and depthwise separable convolution. Dataset: Use a subset of the CIFAR-100 dataset.

```
import numpy as np
import matplotlib.pyplot as
plt import tensorflow as tf
```

```
from tensorflow.keras.datasets import cifar100
from tensorflow.keras.models import
Sequential from tensorflow.keras.layers import
(
   Input, Conv2D, SeparableConv2D,
   MaxPooling2D, GlobalAveragePooling2D,
   Dense
)

# 1. Load CIFAR-100 dataset
(X_train, y_train), (X_test, y_test) = cifar100.load_data(
   label_mode="fine"
)

# Use a subset to reduce training time
np.random.seed(42)

train_indices = np.random.choice(
   len(X_train), 10000, replace=False
)
test_indices = np.random.choice(
   len(X_test), 2000, replace=False
)

X_train =
X_train[train_indices]
```

y_train =

y_train[train_indices]

```
X_test =
X_test[test_indices] y_test
= y_test[test_indices]

print("Training subset:", X_train.shape)
print("Testing subset:", X_test.shape)

# 2. Normalize images
X_train = X_train.astype("float32") / 255.0
X_test = X_test.astype("float32") / 255.0

# 3. Build model with standard
convolution def build_standard_cnn():

  model = Sequential([
    Input(shape=(32, 32,
    3)),

    Conv2D(32, (3, 3), padding="same",
        activation="relu"),
    MaxPooling2D((2, 2)),

    Conv2D(64, (3, 3), padding="same",
        activation="relu"),
    MaxPooling2D((2, 2)),
```

GlobalAveragePooling2D(),

```
    Dense(128, activation="relu"),
    Dense(100,
    activation="softmax")
  ])

  model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy
    ", metrics=["accuracy"]
  )

  return model

# 4. Build model with depthwise separable
convolution def build_separable_cnn():

  model = Sequential([
    Input(shape=(32, 32,
    3)),

    SeparableConv2D(
      32, (3, 3), padding="same",
      activation="relu"
    ),
    MaxPooling2D((2, 2)),
```

```
        SeparableConv2D(
            64, (3, 3), padding="same",
            activation="relu"
        ),
        MaxPooling2D((2, 2)),

        GlobalAveragePooling2D(),

        Dense(128, activation="relu"),
        Dense(100,
        activation="softmax")
    ])

    model.compile(
        optimizer="adam",
        loss="sparse_categorical_crossentropy
        ", metrics=["accuracy"]
    )

    return model

# 5. Train standard convolution model
tf.keras.utils.set_random_seed(42)

standard_model = build_standard_cnn()
```

```
print("\nStandard CNN")
standard_model.summary()

standard_history = standard_model.fit(
   X_train, y_train,
   epochs=5,
   batch_size=128,
   validation_split=0.
   1, verbose=1
)

standard_loss, standard_accuracy = standard_model.evaluate(
   X_test, y_test, verbose=0
)

# 6. Train depthwise separable convolution
model tf.keras.utils.set_random_seed(42)

separable_model = build_separable_cnn()

print("\nDepthwise Separable CNN")
separable_model.summary()

separable_history = separable_model.fit(
   X_train, y_train,
```

```
    epochs=5,
    batch_size=128,
    validation_split=0.
    1, verbose=1
)

separable_loss, separable_accuracy =
    separable_model.evaluate( X_test, y_test, verbose=0
)

# 7. Compare parameters and accuracy
standard_params =
standard_model.count_params()
separable_params =
separable_model.count_params()

print("\nFinal
Comparison") print("   ")
print("Standard Conv Parameters:", standard_params)
print("Separable Conv Parameters:", separable_params)

print(
    "Standard CNN Test Accuracy:",
    round(standard_accuracy * 100, 2), "%"
)
```

print(

```
    "Separable CNN Test Accuracy:",
    round(separable_accuracy * 100, 2), "%"
)

print(
    "Parameter
    Reduction:", round(
        (1 - separable_params / standard_params) *
        100, 2
    ),
    "%"
)

# 8. Plot test accuracy comparison
plt.figure(figsize=(7, 5))

plt.bar(
    ["Standard Conv", "Depthwise Separable"],
    [standard_accuracy * 100, separable_accuracy *
    100]
)

plt.ylabel("Test Accuracy (%)")
plt.title("CIFAR-100 Convolution Comparison")
plt.show()
```

```
# 9. Plot training loss
plt.figure(figsize=(8,
5))

plt.plot(
   standard_history.history["loss"
   ], label="Standard
   Convolution"
)

plt.plot(
   separable_history.history["loss
   "], label="Depthwise
   Separable"
)

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss
Comparison") plt.legend()
plt.grid(True)
plt.show()
```

7. Implement a convolutional autoencoder and use it to extract features from images.

Dataset: Use the STL-10 dataset, which includes unlabeled images.

```
import numpy as np

import matplotlib.pyplot as plt
```

```
import tensorflow as tf
import tensorflow_datasets as tfds

# 1. Load unlabeled STL-10
images dataset = tfds.load(
   "stl10",
   split="unlabelled[:10%]",
   as_supervised=False
)

def preprocess(example):
   image = tf.image.resize(example["image"], (96,
   96)) image = tf.cast(image, tf.float32) / 255.0
   return image

dataset = dataset.map(
   lambda example: (preprocess(example), preprocess(example))
).batch(32).prefetch(tf.data.AUTOTUNE)

# 2. Build convolutional encoder
input_image = tf.keras.Input(shape=(96, 96, 3))

x = tf.keras.layers.Conv2D(
   32, 3, strides=2, padding="same", activation="relu"
)(input_image)
```

```
x = tf.keras.layers.Conv2D(
    64, 3, strides=2, padding="same", activation="relu"
)(x)

x = tf.keras.layers.Conv2D(
    128, 3, strides=2, padding="same", activation="relu"
)(x)

features = tf.keras.layers.Conv2D(
    64, 3, padding="same",
    activation="relu",
    name="encoded_features"
)(x)

encoder = tf.keras.Model(input_image, features, name="encoder")

# 3. Build decoder
x = tf.keras.layers.Conv2DTranspose(
    128, 3, strides=2, padding="same", activation="relu"
)(features)

x = tf.keras.layers.Conv2DTranspose(
    64, 3, strides=2, padding="same", activation="relu"
)(x)
```

```
x = tf.keras.layers.Conv2DTranspose(
    32, 3, strides=2, padding="same", activation="relu"
)(x)

output_image = tf.keras.layers.Conv2D(
    3, 3, padding="same", activation="sigmoid"
)(x)

autoencoder = tf.keras.Model(
    input_image, output_image, name="autoencoder"
)

# 4. Compile model
autoencoder.compile(
    optimizer="adam",
    loss="mse"
)

autoencoder.summary()

# 5. Train autoencoder
history = autoencoder.fit(
    dataset,
    epochs=10
)
```

```
# 6. Get one batch for
visualization for images, _ in
dataset.take(1):
   original_images = images.numpy()
   reconstructed_images = autoencoder.predict(
      images, verbose=0
   )
   extracted_features = encoder.predict(
      images, verbose=0
   )
   break

print("Original image shape:", original_images.shape)
print("Extracted feature shape:", extracted_features.shape)

# 7. Display original and reconstructed
images plt.figure(figsize=(12, 5))

for i in range(5):
   plt.subplot(2, 5, i + 1)
   plt.imshow(original_images
   [i]) plt.title("Original")
   plt.axis("off")

   plt.subplot(2, 5, i + 6)
```

```
    plt.imshow(np.clip(reconstructed_images[i], 0,
    1)) plt.title("Reconstructed")
    plt.axis("off")

plt.tight_layout()
plt.show()

# 8. Visualize one extracted feature map
plt.figure(figsize=(10, 4))

for i in range(6):
    plt.subplot(2, 3, i + 1)
    plt.imshow(extracted_features[0, :, :, i], cmap="viridis")
    plt.title(f"Feature Map {i + 1}")
    plt.axis("off")

plt.tight_layout()
plt.show()

# 9. Plot reconstruction loss
plt.plot(history.history["loss"])
plt.xlabel("Epoch")
plt.ylabel("Reconstruction MSE")
plt.title("Autoencoder Training Loss")
plt.grid(True)
```

plt.show()

8. Use a pre-trained CNN model (e.g., VGG16, ResNet) and fine-tune it for a specific image classification task. Dataset: Use a subset of the ImageNet dataset or a custom dataset relevant to the task.

```
import tensorflow as tf
import tensorflow_datasets as
tfds import matplotlib.pyplot as
plt

from tensorflow.keras.applications import VGG16
from tensorflow.keras.applications.vgg16 import
preprocess_input from tensorflow.keras import layers, Model

# 1. Load flower dataset
(train_ds, val_ds), info =
tfds.load(
    "tf_flowers",
    split=["train[:80%]",
    "train[80%:]"],
    as_supervised=True,
    with_info=True
)

NUM_CLASSES =
info.features["label"].num_classes IMG_SIZE =
```

160

BATCH_SIZE = 32

```
# 2. Preprocess images
def prepare_image(image, label):
   image = tf.image.resize(image, (IMG_SIZE,
   IMG_SIZE)) image = tf.cast(image, tf.float32)
   image =
   preprocess_input(image)
   return image, label

train_ds = (
   train_ds.map(prepare_image,  num_parallel_calls=tf.data.AUTOTUNE)
   .shuffle(500)
   .batch(BATCH_SIZE)
   .prefetch(tf.data.AUTOTUNE)
)

val_ds = (
   val_ds.map(prepare_image,  num_parallel_calls=tf.data.AUTOTUNE)
   .batch(BATCH_SIZE)
   .prefetch(tf.data.AUTOTUNE)
)

# 3. Load pre-trained
VGG16 base_model =
VGG16(
   weights="imagenet",
   include_top=False,
```

```
input_shape=(IMG_SIZE, IMG_SIZE,
3)
```

```
)

# Freeze pretrained layers initially
base_model.trainable = False

# 4. Add classification head
inputs = layers.Input(shape=(IMG_SIZE,
IMG_SIZE, 3)) x = base_model(inputs,
training=False)
x = layers.GlobalAveragePooling2D()(x)
x = layers.Dense(128,
activation="relu")(x) x =
layers.Dropout(0.3)(x)
outputs = layers.Dense(NUM_CLASSES, activation="softmax")(x)

model = Model(inputs, outputs)

model.compile(
   optimizer=tf.keras.optimizers.Adam(learning_rate=0.0
   01), loss="sparse_categorical_crossentropy",
   metrics=["accuracy"]
)

# 5. Train the new classification head
history1 = model.fit(
   train_ds,
```

validation_data=val_ds,

```
    epochs=3
)

# 6. Fine-tune the last VGG16 block
base_model.trainable = True

for layer in base_model.layers:
    if not layer.name.startswith("block5"):
        layer.trainable = False

model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=1
    e-5), loss="sparse_categorical_crossentropy",
    metrics=["accuracy"]
)

history2 = model.fit(
    train_ds,
    validation_data=val_ds,
    epochs=3
)

# 7. Evaluate the fine-tuned model
loss, accuracy = model.evaluate(val_ds, verbose=1)
```

```
print("\nFine-tuned VGG16 Results")

print("Validation Loss:", round(loss, 4))

print("Validation Accuracy:", round(accuracy * 100, 2), "%")


# 8. Plot validation accuracy

val_accuracy = (

  history1.history["val_accuracy"]

  + history2.history["val_accuracy"]

)


plt.plot(range(1, len(val_accuracy) + 1), val_accuracy,

marker="o") plt.xlabel("Epoch")

plt.ylabel("Validation  Accuracy")

plt.title("VGG16 Transfer Learning and Fine-

Tuning") plt.grid(True)

plt.show()
```

9. Implement a simple object detection model using a CNN and train it to detect objects in images. Dataset: Use the PASCAL VOC dataset.

```
from ultralytics import YOLO

import matplotlib.pyplot as

plt from pathlib import Path


# 1. Load a small pretrained YOLO model

model = YOLO("yolov8n.pt")
```

```
# 2. Train on PASCAL VOC
# Ultralytics downloads/prepares the dataset as
needed. results = model.train(
   data="VOC.yaml",
   epochs=10,
   imgsz=320,
   batch=16,
   project="runs/voc_detection
   ", name="yolo_voc",
   exist_ok=True
)

# 3. Validate the trained
model metrics = model.val(
   data="VOC.yaml",
   imgsz=320
)

print("\nObject Detection
Evaluation") print("----------")
print("mAP50:",
float(metrics.box.map50))
print("mAP50-95:",
float(metrics.box.map))
```

4. Predict objects in a sample image

```
# Replace this path with an image from your VOC dataset.
image_path = "sample.jpg"

if Path(image_path).exists():
    predictions =
        model.predict(
        source=image_path,
        imgsz=320,
        conf=0.25,
        save=True
    )

    # Display detection result
    result_image =
    predictions[0].plot()
    plt.figure(figsize=(8, 6))
    plt.imshow(result_image[:, :, ::-
    1])      plt.title("YOLO      Object
    Detection") plt.axis("off")
    plt.show()
else:
    print(
        "\nPlace a test image named sample.jpg in
        the " "current folder to visualize detections."
    )
```

10. Implement an image segmentation model using a CNN (e.g., U-Net) and train it to

segment objects in images. Dataset: Use the COCO dataset for segmentation tasks.

```
from ultralytics import YOLO
import matplotlib.pyplot as
plt from pathlib import Path

# 1. Load a pretrained CNN segmentation
model model = YOLO("yolo11n-seg.pt")

# 2. Select dataset
# coco8-seg.yaml is a small sample dataset for
testing. # Use coco-seg.yaml if supported by your
installed
# Ultralytics version for full COCO segmentation
training. DATASET = "coco8-seg.yaml"

# 3. Train the segmentation model
model.train(
   data=DATASET,
   epochs=10,
   imgsz=320,
   batch=8,
   project="runs/coco_segmentation",
   name="yolo_seg",
   exist_ok=True
)
```

```
# 4. Validate the trained
model metrics = model.val(
    data=DATASET,
    imgsz=320
)

print("\nSegmentation
Evaluation") print("----------")
print("Mask mAP50:", float(metrics.seg.map50))
print("Mask mAP50-95:", float(metrics.seg.map))

# 5. Load the best saved model
best_model_path = Path(
    "runs/coco_segmentation/yolo_seg/weights/best.pt"
)

if best_model_path.exists():
    model = YOLO(str(best_model_path))

# 6. Predict segmentation masks on an image
# Replace sample.jpg with an image of your
choice. image_path = "sample.jpg"

if Path(image_path).exists():
```

```
results =
   model.predict(
   source=image_path
   , imgsz=320,
   conf=0.25,
   save=True
)

# Plot predicted masks and object labels
result_image = results[0].plot()

plt.figure(figsize=(9, 6))
plt.imshow(result_image[:, :, ::-1])
plt.title("CNN Image Segmentation")
plt.axis("off")
plt.show()

# Print number of detected
objects if results[0].masks is not
None:
   print(
      "Objects segmented:",
      len(results[0].masks.data)
   )
else:
   print("No objects were segmented.")
```

```
else:
    print(
        "\nPlace an image named sample.jpg in the "
        "current folder to visualize segmentation."
    )
```